

# Quiz 2 Study Guide — Lectures 4–7

CSCE 775 — Spring 2026

J.C. Vaught

**Quiz format:** 1 math problem (50%) + conceptual questions (50%). Strategy: nail the conceptual questions, get partial credit on the math.

## Contents

|           |                                                       |          |
|-----------|-------------------------------------------------------|----------|
| <b>I</b>  | <b>Lecture 4: Model-Free RL</b>                       | <b>2</b> |
| <b>1</b>  | <b>Big Picture</b>                                    | <b>2</b> |
| <b>2</b>  | <b>Monte Carlo Methods (Ch. 5)</b>                    | <b>2</b> |
| 2.1       | MC Prediction . . . . .                               | 2        |
| 2.2       | MC Control (On-Policy) . . . . .                      | 2        |
| 2.3       | On-Policy vs Off-Policy . . . . .                     | 3        |
| <b>3</b>  | <b>Temporal-Difference Learning (Ch. 6)</b>           | <b>3</b> |
| 3.1       | TD(0) Prediction . . . . .                            | 3        |
| 3.2       | SARSA — On-Policy TD Control . . . . .                | 3        |
| 3.3       | Q-Learning — Off-Policy TD Control . . . . .          | 3        |
| <b>4</b>  | <b><math>n</math>-Step Methods (Ch. 7)</b>            | <b>3</b> |
| <b>5</b>  | <b>Unified View: DP vs TD</b>                         | <b>4</b> |
| <b>II</b> | <b>Lecture 5: Deep Learning Fundamentals</b>          | <b>5</b> |
| <b>6</b>  | <b>Linear Models</b>                                  | <b>5</b> |
| 6.1       | Linear Regression . . . . .                           | 5        |
| 6.2       | Logistic Regression (Binary Classification) . . . . . | 5        |
| 6.3       | Softmax (Multi-class) . . . . .                       | 5        |
| <b>7</b>  | <b>Neural Networks</b>                                | <b>5</b> |
| 7.1       | Architecture . . . . .                                | 5        |
| 7.2       | Common Activation Functions . . . . .                 | 5        |

|            |                                                    |           |
|------------|----------------------------------------------------|-----------|
| 7.3        | Universal Function Approximation . . . . .         | 6         |
| 7.4        | Backpropagation . . . . .                          | 6         |
| 7.5        | Hyperparameters vs Parameters . . . . .            | 6         |
| <b>8</b>   | <b>Overfitting and Regularization</b>              | <b>6</b>  |
| <b>9</b>   | <b>Optimization</b>                                | <b>6</b>  |
| 9.1        | Residual Networks (ResNets) . . . . .              | 6         |
| 9.2        | Batch Normalization . . . . .                      | 7         |
| <b>III</b> | <b>Lecture 6: PyTorch</b>                          | <b>8</b>  |
| <b>10</b>  | <b>Key Concepts (Likely Conceptual Questions)</b>  | <b>8</b>  |
| <b>11</b>  | <b>Training Loop Pattern</b>                       | <b>8</b>  |
| <b>12</b>  | <b>ResNet in PyTorch</b>                           | <b>8</b>  |
| <b>13</b>  | <b>Gradient Checking</b>                           | <b>8</b>  |
| <b>IV</b>  | <b>Lecture 7: DAVI and Deep Q-Networks</b>         | <b>9</b>  |
| <b>14</b>  | <b>Deep Approximate Value Iteration (DAVI)</b>     | <b>9</b>  |
| 14.1       | Target Network . . . . .                           | 9         |
| <b>15</b>  | <b>Deep Q-Networks (DQN)</b>                       | <b>9</b>  |
| 15.1       | Three Key DQN Innovations . . . . .                | 9         |
| 15.2       | DQN Ablation (know this!) . . . . .                | 9         |
| <b>16</b>  | <b>DQN Improvements</b>                            | <b>10</b> |
| <b>V</b>   | <b>Likely Math Problem Targets</b>                 | <b>11</b> |
| <b>17</b>  | <b>Candidate 1: TD(0) or MC Update Calculation</b> | <b>11</b> |
| <b>18</b>  | <b>Candidate 2: Q-Learning or SARSA Update</b>     | <b>11</b> |
| <b>19</b>  | <b>Candidate 3: Gradient Descent Step</b>          | <b>11</b> |
| <b>20</b>  | <b>Candidate 4: DQN / DAVI Target Computation</b>  | <b>11</b> |
| <b>VI</b>  | <b>Rapid-Fire Conceptual Review</b>                | <b>12</b> |

## Part I

# Lecture 4: Model-Free RL

Covers Sutton & Barto Ch. 5.1–5.7, 6.1–6.5, 7.1–7.3

## 1 Big Picture

**Model-free RL** means learning from *experience* without knowing the transition function  $p(s', r \mid s, a)$ .

Two tasks:

- **Prediction (policy evaluation):** estimate  $v_\pi$  or  $q_\pi$  for a given  $\pi$
- **Control:** find the optimal policy  $\pi_*$

Three families of methods, each with a different *target* for the update:

| Method       | Target                                                         | Bias            | Variance |
|--------------|----------------------------------------------------------------|-----------------|----------|
| Monte Carlo  | $G_t$ (full return)                                            | None            | High     |
| TD(0)        | $R_{t+1} + \gamma V(S_{t+1})$                                  | Yes (bootstrap) | Low      |
| $n$ -step TD | $R_{t+1} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$ | Medium          | Medium   |

## 2 Monte Carlo Methods (Ch. 5)

### 2.1 MC Prediction

Estimate  $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$  by averaging *sampled returns*.

MC Incremental Update:  $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$

- **First-visit MC:** average  $G_t$  only the *first* time  $s$  appears in an episode.
- **Every-visit MC:** average  $G_t$  every time  $s$  appears.
- Must wait until the episode ends to compute  $G_t$ . Does *not* work for continuing (non-episodic) tasks.

### 2.2 MC Control (On-Policy)

Use  $Q(s, a)$  instead of  $V(s)$  because we cannot derive a policy from  $V$  without a model.

Policy from Q:  $\pi(s) = \arg \max_a Q(s, a)$

**$\varepsilon$ -greedy exploration:**

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & \text{if } a = \arg \max_{a'} Q(s, a') \\ \varepsilon/|\mathcal{A}(s)| & \text{otherwise} \end{cases}$$

The on-policy MC control algorithm: generate episode  $\rightarrow$  compute returns backward  $\rightarrow$  update  $Q(S_t, A_t)$   $\rightarrow$  improve policy to  $\varepsilon$ -greedy w.r.t. new  $Q$ .

## 2.3 On-Policy vs Off-Policy

- **Behavior policy  $b$ :** the policy used to collect data.
- **Target policy  $\pi$ :** the policy we want to evaluate/improve.
- **On-policy:**  $b = \pi$ .
- **Off-policy:**  $b \neq \pi$ . Can reuse data from other policies. Requires importance sampling.

## 3 Temporal-Difference Learning (Ch. 6)

### 3.1 TD(0) Prediction

**Key idea:** update toward a *bootstrapped* target instead of waiting for  $G_t$ .

$$\text{TD(0) Update: } V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD target}}$$

The quantity  $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$  is the **TD error**.

**MC vs TD:**

- MC: unbiased, high variance, needs full episodes.
- TD: biased (initially), low variance, learns online step-by-step.
- TD bootstraps from its own estimate  $\Rightarrow$  can propagate value information faster.

### 3.2 SARSA — On-Policy TD Control

$$\text{SARSA Update: } Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Name comes from the quintuple  $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ .

Both behavior and target policy are  $\varepsilon$ -greedy. Choose  $A_{t+1}$  from  $S_{t+1}$  using  $\varepsilon$ -greedy *before* the update.

### 3.3 Q-Learning — Off-Policy TD Control

$$\text{Q-Learning Update: } Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

**Key difference:** uses  $\max_a Q(S_{t+1}, a)$  (greedy) instead of  $Q(S_{t+1}, A_{t+1})$  ( $\varepsilon$ -greedy).

- Behavior policy:  $\varepsilon$ -greedy (explores).
- Target policy: greedy (deterministic).
- Off-policy because behavior  $\neq$  target.

**Cliff Walking Example:**

- SARSA learns a *safer* path (avoids cliff) because it accounts for its own  $\varepsilon$ -greedy exploration.
- Q-learning learns the *optimal* path (close to cliff) because the target policy is greedy.

## 4 $n$ -Step Methods (Ch. 7)

$$n\text{-step return: } G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- $n = 1$ : TD(0).
- $n = \infty$ : Monte Carlo.
- Typical sweet spot:  $n \approx 5$ .

**TD( $\lambda$ )** averages over *all*  $n$ -step returns with exponentially decaying weights  $(1 - \lambda)\lambda^{n-1}$ . When  $\lambda = 0$  you get TD(0); when  $\lambda = 1$  you get MC.

## 5 Unified View: DP vs TD

|                                 | Full Backup (DP)   | Sample Backup (TD) |
|---------------------------------|--------------------|--------------------|
| Bellman Expectation for $v_\pi$ | Policy Evaluation  | TD Learning        |
| Bellman Expectation for $q_\pi$ | Q-Policy Iteration | SARSA              |
| Bellman Optimality for $q_*$    | Q-Value Iteration  | Q-Learning         |

**Key difference:** DP uses the full model (all transitions); TD samples one transition.

## Part II

# Lecture 5: Deep Learning Fundamentals

## 6 Linear Models

### 6.1 Linear Regression

$$f(\mathbf{x}, \mathbf{w}, b) = \mathbf{w}^\top \mathbf{x} + b \quad \text{Loss: } \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (y_i - f(\mathbf{x}_i, \boldsymbol{\theta}))^2$$

Analytical solution:  $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Gradient descent update:  $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$

### 6.2 Logistic Regression (Binary Classification)

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1+e^{-z}} \quad \text{Derivative: } \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$ . Decision boundary where  $\mathbf{w}^\top \mathbf{x} = 0$ .

Loss: negative log-likelihood (cross-entropy):

$$\mathcal{L} = - \sum_i [y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i))]$$

### 6.3 Softmax (Multi-class)

$$P(y = i \mid \mathbf{x}) = \frac{e^{\mathbf{w}_i^\top \mathbf{x}}}{\sum_{j=1}^C e^{\mathbf{w}_j^\top \mathbf{x}}}$$

## 7 Neural Networks

### 7.1 Architecture

A 2-layer network:  $f(\mathbf{x}) = \mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x})$

**Activation functions must be non-linear**, otherwise the network collapses to a linear model regardless of depth.

### 7.2 Common Activation Functions

| Name       | Formula                      | Notes                                   |
|------------|------------------------------|-----------------------------------------|
| Sigmoid    | $\sigma(x) = 1/(1 + e^{-x})$ | Output in $(0, 1)$ ; vanishing gradient |
| Tanh       | $\tanh(x)$                   | Output in $(-1, 1)$                     |
| ReLU       | $\max(0, x)$                 | Most common; dead neurons possible      |
| Leaky ReLU | $\max(0.1x, x)$              | Fixes dead neuron problem               |

### 7.3 Universal Function Approximation

With enough hidden units, a neural network can approximate *any* continuous function to arbitrary precision. Does *not* guarantee convergence of training.

### 7.4 Backpropagation

Chain rule applied layer by layer. For output layer weights:

$$\frac{\partial E}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i) h_j$$

For hidden layer weights, errors propagate backward through the chain rule.

### 7.5 Hyperparameters vs Parameters

- **Parameters:** learned from data (weights, biases).
- **Hyperparameters:** set before training (learning rate, # layers, # neurons,  $\varepsilon$ , etc.).

## 8 Overfitting and Regularization

| Technique             | How it works                              |
|-----------------------|-------------------------------------------|
| Train/Val/Test split  | Monitor val loss; never train on test     |
| Weight regularization | Add $\lambda \sum W_{ij}^2$ to loss       |
| Dropout               | Randomly zero out neurons during training |
| Data augmentation     | Rotations, flips, noise to increase data  |
| Early stopping        | Stop when validation error starts rising  |

## 9 Optimization

Loss surface is non-convex: local minima, saddle points.

| Optimizer      | Update Rule                                                                                                   |
|----------------|---------------------------------------------------------------------------------------------------------------|
| SGD            | $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla E$                                                          |
| SGD + Momentum | $\mathbf{v} \leftarrow \mu \mathbf{v} + \alpha \nabla E; \quad \mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}$ |
| ADAM           | Adaptive lr per parameter using mean & variance of gradients                                                  |

**Stochastic GD:** compute gradients on random mini-batches instead of full dataset.

### 9.1 Residual Networks (ResNets)

Skip connections:  $\text{output} = F(\mathbf{x}) + \mathbf{x}$ . Smooths the loss surface; enables training very deep networks (100+ layers).

## 9.2 Batch Normalization

Normalize activations to mean 0, std 1 within each mini-batch. Stabilizes training, allows larger learning rates.

## Part III

# Lecture 6: PyTorch

## 10 Key Concepts (Likely Conceptual Questions)

1. **Automatic differentiation:** define forward pass; PyTorch computes backward pass (gradients) automatically via computation graph.
2. **Tensor:** multi-dimensional array (like numpy ndarray), but tracks gradients.
3. **Computation graph:** directed graph of operations. Forward pass builds it; `loss.backward()` traverses it backward.
4. `.detach()`: removes a tensor from the computation graph (stops gradient flow).
5. **Data type:** PyTorch expects `float32`. Convert before or after creating tensors.
6. `loss.item()`: extract scalar value without keeping the computation graph (prevents memory leak).
7. `nnet.train()` vs `nnet.eval()`: training mode enables dropout and batch norm updates; eval mode disables them.
8. **Saving/loading:** `torch.save(nnet.state_dict(), path)` saves only parameters, not architecture.

## 11 Training Loop Pattern

1. `optimizer.zero_grad()` — clear old gradients
2. Forward pass: `output = nnet(input)`
3. Compute loss: `loss = criterion(output, target)`
4. Backward pass: `loss.backward()`
5. Update weights: `optimizer.step()`

## 12 ResNet in PyTorch

Residual block: two linear layers + skip connection. Requires input dim = hidden dim (or use a linear projection). Forward:  $\mathbf{x} \leftarrow \text{ReLU}(F(\mathbf{x}) + \mathbf{x})$ .

## 13 Gradient Checking

Finite differences:  $\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta_i + h) - f(\theta_i)}{h}$  for small  $h$  (e.g.,  $10^{-5}$ ). Verify each layer independently, then compose.

## Part IV

# Lecture 7: DAVI and Deep Q-Networks

## 14 Deep Approximate Value Iteration (DAVI)

When the state space is too large for a table, approximate  $v_*(s)$  with a neural network  $v_\theta(s)$ .

$$\text{DAVI Target: } y = \begin{cases} \max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_\theta(s')) & \text{if } s \text{ non-terminal} \\ 0 & \text{if } s \text{ terminal} \end{cases}$$

$$\text{Loss: } L(\theta) = \frac{1}{2}(y - v_\theta(s))^2 \quad \text{Gradient: } \nabla_\theta L = (y - v_\theta(s)) \nabla_\theta v_\theta(s)$$

**Do NOT differentiate  $y$  with respect to  $\theta$**  even though  $y$  depends on  $\theta$ . Treat  $y$  as a fixed target.

### 14.1 Target Network

Problem: target  $y$  changes every iteration (non-stationary).

Solution: maintain a separate **target network**  $v_{\theta^-}$  whose parameters are periodically copied from  $\theta$ . Use  $\theta^-$  to compute  $y$ , train  $\theta$ .

## 15 Deep Q-Networks (DQN)

Approximate  $q_*(s, a)$  with a neural network. Input: state  $s$ . Output:  $Q$ -values for all actions (one forward pass).

$$\text{DQN Target: } y = \begin{cases} r + \gamma \max_{a'} q_{\theta^-}(s', a') & \text{non-terminal} \\ r & \text{terminal} \end{cases}$$

$$\text{DQN Loss: } L(\theta) = \frac{1}{2}(y - q_\theta(s, a))^2$$

### 15.1 Three Key DQN Innovations

1. **Target network** ( $\theta^-$  updated periodically) — stabilizes training by making targets less non-stationary.
2. **Experience replay buffer** — store  $(s, a, r, s')$  tuples; sample random mini-batches to decorrelate training data and prevent catastrophic forgetting.
3. **Huber loss** — clips gradients for large TD errors, preventing instability from large rewards.

### 15.2 DQN Ablation (know this!)

Both replay buffer AND target network are essential. Removing either one drastically hurts performance. DQN also vastly outperforms linear function approximators.

## 16 DQN Improvements

| Method                    | Key Idea                                                                                                                                                                                           |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Double Q-Learning</b>  | Two Q-networks: one selects the action, the other evaluates it. Fixes <i>maximization bias</i> . $y = r + \gamma q_{\theta_2}(s', \arg \max_{a'} q_{\theta_1}(s', a'))$                            |
| <b>Dueling DQN</b>        | Split network into value stream $v_{\theta}(s)$ and advantage stream $A_{\theta}(s, a)$ . $q(s, a) = v_{\theta}(s) + A_{\theta}(s, a)$ . Advantage: $A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$ . |
| <b>Prioritized Replay</b> | Sample transitions with large TD error more frequently.                                                                                                                                            |
| <b>Rainbow</b>            | Combines DQN + Double + Dueling + Prioritized + others. Best overall.                                                                                                                              |
| <b>Agent57</b>            | First to beat humans on all 57 Atari games. Uses intrinsic curiosity + adaptive discount.                                                                                                          |

## Part V

# Likely Math Problem Targets

The math problem is worth 50%. Based on the material, the most likely candidates are:

### 17 Candidate 1: TD(0) or MC Update Calculation

Given a small MDP or sequence of transitions, compute the updated value after one or more steps.

**Example:** Given  $V(A) = 0$ ,  $V(B) = 0.5$ ,  $\gamma = 1$ ,  $\alpha = 0.1$ , and the transition  $A \xrightarrow{r=0} B$ :

$$V(A) \leftarrow 0 + 0.1[0 + 1 \cdot 0.5 - 0] = 0.05$$

### 18 Candidate 2: Q-Learning or SARSA Update

**Example:** Given  $Q(s, \text{left}) = 1$ ,  $Q(s, \text{right}) = 2$ ,  $Q(s', \text{left}) = 3$ ,  $Q(s', \text{right}) = 5$ ,  $\alpha = 0.5$ ,  $\gamma = 0.9$ ,  $r = -1$ , action taken = left:

**Q-learning:**

$$Q(s, \text{left}) \leftarrow 1 + 0.5[-1 + 0.9 \cdot \max(3, 5) - 1] = 1 + 0.5[-1 + 4.5 - 1] = 1 + 1.25 = 2.25$$

**SARSA** (if next action = left):

$$Q(s, \text{left}) \leftarrow 1 + 0.5[-1 + 0.9 \cdot 3 - 1] = 1 + 0.5[0.7] = 1.35$$

### 19 Candidate 3: Gradient Descent Step

**Example:** Given  $f(x, w) = wx$ , data point  $(x = 2, y = 5)$ , current  $w = 1$ ,  $\alpha = 0.1$ :

$$\mathcal{L} = \frac{1}{2}(5 - 2)^2 = 4.5, \quad \frac{\partial \mathcal{L}}{\partial w} = -(5 - 2) \cdot 2 = -6, \quad w \leftarrow 1 - 0.1(-6) = 1.6$$

### 20 Candidate 4: DQN / DAVI Target Computation

**Example:** Compute the DQN target given:  $r = 3$ ,  $\gamma = 0.99$ ,  $q_{\theta^-}(s', a_1) = 10$ ,  $q_{\theta^-}(s', a_2) = 15$ :

$$y = 3 + 0.99 \cdot \max(10, 15) = 3 + 14.85 = 17.85$$

Then loss:  $L = \frac{1}{2}(17.85 - q_{\theta}(s, a))^2$

## Part VI

# Rapid-Fire Conceptual Review

1. **Why Q instead of V for model-free control?** Cannot derive  $\pi$  from  $V(s)$  without knowing  $p(s'|s, a)$ . With  $Q(s, a)$ , just take  $\arg \max_a$ .
2. **MC vs TD bias/variance?** MC: unbiased, high variance. TD: biased (bootstraps), low variance.
3. **Can MC handle continuing tasks?** No. Must wait for episode to end.
4. **SARSA vs Q-learning?** SARSA is on-policy (uses  $Q(S', A')$ ); Q-learning is off-policy (uses  $\max_a Q(S', a)$ ). In cliff walking, SARSA is safer, Q-learning is optimal.
5. **What is bootstrapping?** Using current estimates of value to update other estimates (TD does this; MC does not).
6. **What is the TD error?**  $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ .
7. **Why use  $\varepsilon$ -greedy?** Ensures exploration. Pure greedy can get stuck.
8. **Why must activations be non-linear?** Otherwise, any depth of network collapses to a single linear transformation.
9. **What is ReLU?**  $\max(0, x)$ . Simple, efficient, avoids vanishing gradient (for  $x > 0$ ).
10. **What is a residual connection?**  $\text{output} = F(\mathbf{x}) + \mathbf{x}$ . Enables training deep networks by smoothing the loss surface.
11. **What is batch normalization?** Normalize layer inputs to mean 0, std 1 per mini-batch. Stabilizes and accelerates training.
12. **Overfitting solutions?** Regularization, dropout, data augmentation, early stopping, simpler model.
13. **Why experience replay?** Decorrelates training samples. Prevents catastrophic forgetting. Improves data efficiency.
14. **Why target network?** Stabilizes training by keeping the target fixed for many updates, reducing non-stationarity.
15. **What is maximization bias?** Q-learning overestimates Q-values because it uses  $\max$  for both selection and evaluation. Double Q-learning fixes this.
16. **What is the advantage function?**  $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$ . How much better action  $a$  is than average.
17. **Dueling DQN?** Separate streams for  $v(s)$  and  $A(s, a)$ , combined as  $q(s, a) = v(s) + A(s, a)$ .
18. **What does `.detach()` do in PyTorch?** Removes a tensor from the computation graph, stopping gradient flow.
19. **`nnet.train()` vs `nnet.eval()`?** Train mode enables dropout and batch norm updates. Eval mode disables them for deterministic inference.
20. **Why `loss.item()` not `loss`?** Using `loss` keeps the computation graph in memory, causing a memory leak.
21. **Universal approximation theorem?** A neural network with enough hidden units can approximate any continuous function. Does NOT guarantee we can find the right weights via training.
22. **SGD vs full gradient descent?** SGD uses random mini-batches instead of full dataset. Faster per

step, noisier gradients, but often converges well.

- 23. **What is  $n$ -step TD?** Intermediate between TD(0) and MC. Uses  $n$  actual rewards then bootstraps.  $n = 1$  is TD(0),  $n = \infty$  is MC.
- 24. **DQN ablation key finding?** Both replay buffer AND target network are essential. Removing either one drastically hurts performance.
- 25. **Robbins-Monro conditions?** For convergence:  $\sum \alpha_n = \infty$  (steps large enough to overcome initial conditions) and  $\sum \alpha_n^2 < \infty$  (steps shrink enough to converge).